



Reporte Técnico de Actividades Práctico-Experimentales Nro. 001

1. Datos de Identificación del Estudiante y la Práctica

Nombre del estudiante(s)	Yimmy Onner Angulo Torres
Asignatura	Teoría de la programación
Ciclo	1 A
Unidad	3
Resultado de aprendizaje de la unidad	Desarrolla aplicaciones utilizando el principio de la programación modular y estructuras de datos simples y/o estáticas compuestas, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	001
Tipo	Individual o Grupal
Título de la Práctica	Construcción de funciones y procedimientos en un lenguaje de programación.
Nombre del Docente	Lisette Geoconda López Faicán
Fecha	Jueves 8 de enero del 2026 Jueves 15 de enero del 2026
Horario	10h30 – 13h30
Lugar	Aula física asignada al paralelo.
Tiempo planificado en el Sílabo	6 horas

2. Objetivo(s) de la Práctica

Escriba el objetivo de la práctica (copiado de la guía proporcionada por el docente).

Aplicar los fundamentos de la programación modular mediante la construcción y uso de funciones y procedimientos, para resolver un problema real, garantizando un código estructurado, reutilizable y correctamente documentado.

3. Materiales Reactivos. Equipos y Herramientas

Liste los materiales, reactivos, equipos y herramientas utilizados en la práctica, confirmando los de la guía o agregando los adicionales.

- IDE de programación: Visual Studio Code u otro entorno compatible.
- Lenguaje de programación: C (según los contenidos de la unidad).

4. Equipos y Herramientas

- Computador personal con sistema operativo Windows, Linux o macOS.
- Material de apoyo en el Entorno Virtual de Aprendizaje (EVA).
- Editores de texto (Word, Google Docs u otros) para la elaboración del informe técnico en formato PDF.
- Conexión a internet estable para acceder a recursos digitales y software en línea.
- Aula física asignada al paralelo.

5. Procedimiento / Metodología Ejecutada

Desarrollo

Para llegar a la solución de este problema, diseñaremos un algoritmo que sea capaz de resolver este problema planteado, con su posterior implementación en un lenguaje de programación de alto nivel, que en esta ocasión es el lenguaje C.

Para realizar esto, se seguirán los siguientes pasos

Codificación: En este apartado, se llevará, todo lo elaborado en el diseño del algoritmo, a un lenguaje de programación de alto nivel en este caso el Lenguaje C, con el manejo del editor de código o IDE conocido como Visual Studio Code. Teniendo una mayor facilidad al momento de escribir el código, corrigiendo errores de indentación, o malas prácticas ejecutadas en el código.

Pruebas: Para las pruebas de escritorio, se las realizó mediante una tabla para verificar que todo funcione de buena manera. Se utilizó la terminal o cmd, incluida dentro del IDE, para comprobar a tiempo real como se ejecuta el código, ayudando a depurar errores existentes y garantizando un código limpio y funcional.

6. Resultados

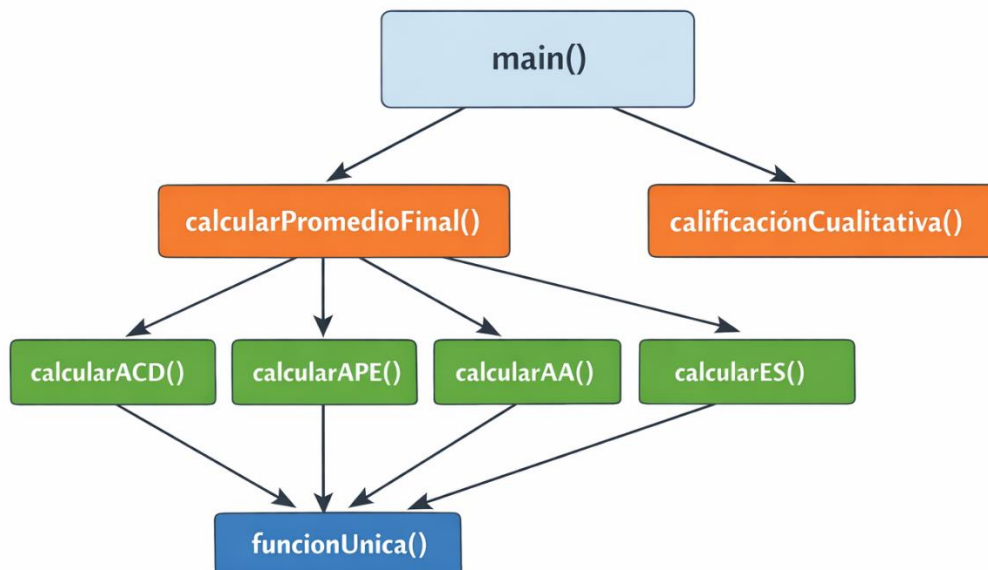
Contextualización del problema: Se requiere desarrollar un programa que calcule la nota final de un estudiante, aplicando funciones y procedimientos para cada componente evaluativo:

- Calcular nota del Aprendizaje en Contacto con el Docente (ACD): solicita nro de actividades, las notas para cada actividad, calcula promedio, y retorna el total ponderado sobre 2.0.
- Calcular nota del Aprendizaje Práctico Experimental (APE): solicita nro de actividades, las notas para cada actividad, calcula promedio, y retorna el total ponderado sobre 2.5.
- Calcular nota del Aprendizaje Autónomo (AA): solicita nro de actividades, las notas para cada actividad, calcula promedio, y retorna el total ponderado sobre 2.0.
- Calcular Evaluación sumativa (ES): solicita la nota de Aprendizaje Basado en Problemas y Portafolio Digital, calcular el ponderado (60% y 40%), y retornar el total ponderado sobre 3.5.
- Calcular promedio por unidad: $ACD + APE + AA + ES$ (retorna el promedio para una unidad).
- Calcular el promedio de la asignatura: promedio simple de acuerdo al número de unidades con la escala cualitativa.
 - ♦ APROBADO: Si la nota final es mayor o igual a 7.
 - ♦ SUPLETORIO: Si la nota final es mayor o igual a 2.5 y menor a 7.
 - ♦ REPROBADO: Si la nota final es menor a 2.5.

Las notas deben estar en el rango de 0.0 a 10.0

- Salida de Resultados: Imprimir la nota cuantitativa y cualitativa final del estudiante.
- Requerimiento del código: El programa debe estar bien estructurado, con comentarios y mensajes descriptivos que faciliten su uso.

Diagrama de las llamadas de las diferentes funciones, para el funcionamiento del programa.



Codificación de la función Main(), la misma que llama a 2 funciones más las cuales son calcularPromedioFinal() y calificaciónCualitativa()

```
#include <stdio.h>

float calcularPromedioFinal(int nu, int j);
float calcularES(int i);
float calcularAA(int i);
float calcularAPE(int i);
float calcularACD(int i);
float funcionUnica(int nac);
char *calificacionCualitativa(float not);
int main()
{
    int contadorEstudiantes = 1;
    int i = 1;
    int confirmacion = 1;

    while (confirmacion == 1)
    {
        int num = 3;
        float nota_final = calcularPromedioFinal(num,
contadorEstudiantes);
```

```
char *escala = calificacionCualitativa(nota_final);
contadorEstudiantes += i;

printf("\n===== RESULTADOS DEL PRIMER CICLO =====\n");
printf("\nLa Calificacion Cuantitativa del Estudiante %i es:
%.1f", contadorEstudiantes - 1, nota_final);
printf("\nLa Calificacion Cualitativa del Estudiante es: %s",
escala);
printf("\nDesea agregar a otro estudiante? (1 = Si y 0 = NO): ");
scanf("%i", &confirmacion);
}
return 0;
}
```

Codificación de la función calcularPromedioFinal() que a su vez llama, a las funciones CalcularACD(), CalcularAA(), CalcularAPE() y CalcularES()

```
float calcularPromedioFinal(int nu, int j)
{
    int i = 1;
    float nota_final = 0;

    printf("\n===== Estudiante %i =====",
j);
    j++;
    for (i = 1; i <= nu; i++)
    {
        float nota_unidad = calcularACD(i) + calcularAPE(i) +
calcularAA(i) + calcularES(i);
        nota_final += nota_unidad;
        printf("\nLa nota de la Unidad %i es: %.2f\n", i, nota_unidad);
    }
    nota_final = nota_final / 3;
    return nota_final;
}
```

Codificación de la función calcularACD(), y esta llama a otra función la cual es funcionUnica() y todas las demás funciones para calcular los promedios de AA, APE llaman a la misma función para realizar los diferentes cálculos.

```
float calcularACD(int i)
{
    int nac = 1, j;
    float notaACD = 0, notaAcumulativa = 0, notaPromedio = 0,
    notaPromedio2;

    printf("\n===== Unidad %i =====\n", i);
    do
    {
        printf("\nCuantas actividades tiene en el ACD de la Unidad %i\n",
i);

        scanf("%i", &nac);
        getchar();

        if (nac <= 0 || nac > 10)
        {
            printf("Cantidad de Actividades Invalida: \n");
        }

    } while (nac <= 0 || nac > 10);

    notaPromedio = funcionUnica(nac);
    notaPromedio2 = notaPromedio * 0.2;

    printf("\nLa nota del ACD de la Unidad %i es: %.2f\n", i,
notaPromedio2);

    return notaPromedio2;
}
```

Codificación de la función CalcularAPE()

```
float calcularAPE(int i)
{
    int nac = 1, j = 1;
    float notaAPE = 0, notaAcumulativa = 0, notaPromedio = 0,
    notaPromedio2;

    do
    {
        printf("\nCuantas actividades tiene en el APE de la Unidad %i\n",
i);
        scanf("%i", &nac);
        getchar();

        if (nac <= 0 || nac > 10)
        {
            printf("Cantidad de Actividades Invalida: \n");
        }
    } while (nac <= 0 || nac > 10);

    notaPromedio = funcionUnica(nac);

    notaPromedio2 = notaPromedio * 0.25;

    printf("\nLa nota del APE de la Unidad %i es: %.2f\n", i,
    notaPromedio2);
    return notaPromedio2;
}
```

Codificación de la función CalcularAA()

```
float calcularAA(int i)
{
    int nac = 1, j = 1;
    float notaAA = 0, notaAcumulativa = 0, notaPromedio = 0,
    notaPromedio2;

    do
    {
        printf("\nCuantas actividades tiene en el AA de la Unidad %i\n",
i);

        scanf("%i", &nac);
        getchar();

        if (nac <= 0 || nac > 10)
        {
            printf("Cantidad de Actividades Invalida: \n");
        }
    }
```

```
    } while (nac <= 0 || nac > 10);

    notaPromedio = funcionUnica(nac);

    notaPromedio2 = notaPromedio * 0.2;

    printf("\nLa nota del AA de la Unidad %i es: %.2f\n", i,
    notaPromedio2);
    return notaPromedio2;
}
```

Codificación de la función CalcularES(), para calcular la nota de la evaluación sumativa de las diferentes unidades, repartidos en ABP y PD

```
float calcularES(int i)
{
    float ev_ABP1, port, ES, PORF, ESF;
    do
    {
        printf("\nIngrese la nota de la Evaluacion ABP de la Unidad %i:",
i);

        scanf("%f", &ev_ABP1);
        if (ev_ABP1 < 0.0 || ev_ABP1 > 10.0)
        {
            printf("La nota ingresada es erronea:\n");
        }
    } while (ev_ABP1 < 0.0 || ev_ABP1 > 10.0);

    do
    {
        printf("\nIngrese la nota del Portafolio de la Unidad %i: ", i);
        scanf("%f", &port);
        if (port < 0 || port > 10)
        {
            printf("La nota ingresada es erronea:\n");
        }
    } while (port < 0 || port > 10);

    ES = (ev_ABP1 / 10) * 6;
    PORF = (port / 10) * 4;
    ESF = (ES + PORF) * 0.35;

    printf("\nLa nota del ES de la Unidad %i es: %.2f\n", i, ESF);
    return ESF;
}
```

Codificación de la función `funcionUnica()`, esta función calcula la nota acumulativa de ACD, APE y AA, y devuelve su valor, para ser calculado en las diferentes funciones dependiendo de su valor ponderado

```
float funcionUnica(int nac){
    int j = 1;
    float notaAcumulativa = 0, nota = 0;
    float notaPromedio = 0;
    do
    {
        do
        {
            printf("Ingrese la nota de la Actividad %i: ", j);
            scanf("%f", &nota);

            if (nota < 0 || nota > 10)
            {
                printf("La nota ingresada es erronea:\n");
                j--;
                notaAcumulativa -= nota;
            }

            notaAcumulativa += nota;
            j++;
        } while (nota < 0 || nota > 10);
    } while (j <= nac);

    notaPromedio = notaAcumulativa / nac;
    return notaPromedio;
}
```

Codificación de la función `calificaciónCualitativa()`, la misma que es llamada por la función `int Main`, para presentar la nota Cualitativa del estudiante dependiendo de la nota final de la asignatura.

```
char *calificacionCualitativa(float not_final){
    char *escala;
    if (not_final >= 7)
    {
        return escala = "APROBADO";
    }
    else if (not_final >= 2.5 && not_final < 7)
    {
        return escala = "SUPLETORIO";
    }
    else
    {
        return escala = "REPROBADO";
    }
}
```



Prueba de Escritorio del código.

```
PS C:\Users\HP\Desktop\Lenguaje C\Unidad3\Calculo Asignatura> .\main.exe

===== Estudiante 1 =====
===== Unidad 1 =====

Cuantas actividades tiene en el ACD de la Unidad 1
1
Ingrese la nota de la Actividad 1: 10

La nota del ACD de la Unidad 1 es: 2.00

Cuantas actividades tiene en el APE de la Unidad 1
1
Ingrese la nota de la Actividad 1: 10

La nota del APE de la Unidad 1 es: 2.50

Cuantas actividades tiene en el AA de la Unidad 1
1
Ingrese la nota de la Actividad 1: 10

La nota del AA de la Unidad 1 es: 2.00

Ingrese la nota de la Evaluacion ABP de la Unidad 1: 10

Ingrese la nota del Portafolio de la Unidad 1: 10

La nota del ES de la Unidad 1 es: 3.50

La nota de la Unidad 1 es: 10.00
```

```
===== Unidad 2 =====

Cuantas actividades tiene en el ACD de la Unidad 2
1
Ingrese la nota de la Actividad 1: 10

La nota del ACD de la Unidad 2 es: 2.00

Cuantas actividades tiene en el APE de la Unidad 2
1
Ingrese la nota de la Actividad 1: 10

La nota del APE de la Unidad 2 es: 2.50

Cuantas actividades tiene en el AA de la Unidad 2
1
Ingrese la nota de la Actividad 1: 10

La nota del AA de la Unidad 2 es: 2.00

Ingrese la nota de la Evaluacion ABP de la Unidad 2: 10

Ingrese la nota del Portafolio de la Unidad 2: 10

La nota del ES de la Unidad 2 es: 3.50

La nota de la Unidad 2 es: 10.00
```



===== Unidad 3 =====

Cuántas actividades tiene en el ACD de la Unidad 3

1

Ingrese la nota de la Actividad 1: 10

La nota del ACD de la Unidad 3 es: 2.00

Cuántas actividades tiene en el APE de la Unidad 3

1

Ingrese la nota de la Actividad 1: 10

La nota del APE de la Unidad 3 es: 2.50

Cuántas actividades tiene en el AA de la Unidad 3

1

Ingrese la nota de la Actividad 1: 10

La nota del AA de la Unidad 3 es: 2.00

Ingrese la nota de la Evaluación ABP de la Unidad 3: 10

Ingrese la nota del Portafolio de la Unidad 3: 10

La nota del ES de la Unidad 3 es: 3.50

La nota de la Unidad 3 es: 10.00

===== RESULTADOS DEL PRIMER CICLO =====

La Calificación Cuantitativa del Estudiante 1 en la Asignatura de Teoría de la Programación es: 10.0

La Calificación Cualitativa del Estudiante es: APROBADO

Desea agregar a otro estudiante? (1 = Si y 0 = NO):

7. Preguntas de Control

- ¿Cuál es la diferencia entre una función y un procedimiento?

Una función es un bloque de código que realiza una tarea específica y retorna un valor al programa principal, mientras que un procedimiento ejecuta una serie de instrucciones, pero no devuelve ningún valor. En el programa desarrollado, las funciones como calcularACD, calcularAPE y calcularPromedioFinal devuelven resultados numéricos, por lo que son funciones propiamente dichas; mientras que un procedimiento solo ejecutaría acciones sin producir un resultado directo.

- ¿Qué ventajas aporta dividir un programa en funciones (modularidad)?

Algunas de las ventajas que ofrece la modularidad son:

Dividir un programa en funciones permite organizar el código de manera ordenada, facilitando su lectura, comprensión y mantenimiento. La modularidad hace posible reutilizar código, detectar errores más rápidamente, realizar mejoras sin afectar otras partes del programa y trabajar en cada componente del sistema de forma independiente, lo que aumenta la calidad y eficiencia del desarrollo.

- ¿Qué se mejoraría del programa si se tuviera que usarlo para varios estudiantes?

Para manejar múltiples estudiantes de manera más eficiente, se podría incorporar el uso de estructuras (struct) o archivos para almacenar la información de cada estudiante. Esto permitiría conservar los datos, generar reportes, consultar calificaciones posteriormente y evitar el ingreso repetitivo de información, haciendo el programa más escalable y funcional para un entorno académico real.

8. Conclusiones

- La implementación del programa permitió aplicar correctamente el principio de programación modular, dividiendo el problema en funciones específicas para cada componente de evaluación (ACD, APE, AA y ES), lo cual facilitó el desarrollo, comprensión y mantenimiento del código.
- El sistema de validación de datos implementado garantiza que las calificaciones ingresadas se encuentren dentro del rango permitido (0.0 a 10.0), asegurando así la confiabilidad de los resultados y evitando errores en el cálculo del promedio final del estudiante.
- El uso de funciones independientes para el cálculo de promedios y la calificación cualitativa permitió obtener una solución estructurada, ordenada y reutilizable, cumpliendo con el objetivo de construir un programa claro, funcional y correctamente documentado.



9. Recomendaciones

- Se recomienda continuar fortaleciendo el uso de la programación modular en futuros proyectos, incorporando estructuras de datos más avanzadas como arreglos y estructuras (struct), con el fin de optimizar la organización del código y facilitar la gestión de información de múltiples estudiantes.
- Se sugiere implementar el almacenamiento de datos en archivos para conservar los registros de calificaciones, lo que permitiría generar reportes automáticos, realizar consultas posteriores y mejorar la funcionalidad del sistema en un entorno académico real.
- Es recomendable mejorar la interfaz del programa mediante mensajes más claros y validaciones adicionales, garantizando una mejor experiencia de usuario y reduciendo posibles errores durante el ingreso de datos.